VittSaathi Possible Questions

1. Most important questions

These are the questions I would prepare first:

1. Explain VittSaathi in 1–2 minutes.
2. What problem does VittSaathi solve?
3. What is the complete architecture of VittSaathi?
4. Explain the end-to-end request flow.
5. Why did you choose Spring Boot?
6. Why did you use React + Spring Boot?
7. Why PostgreSQL?
8. Why Redis?
9. Why JWT authentication?
10. How does your JWT authentication work?
11. Why did you use Docker?
12. Why did you use LangGraph?
13. Why multiple agents instead of one LLM agent?
14. What are Sentinel, Kalpa and Drishti responsible for?
15. How do the three agents communicate?
16. What is StateGraph?
17. What is RAG?
18. Why did you need RAG in VittSaathi?
19. How does the system access private user data?
20. How do you ensure private financial data is secure?

Your resume specifically states that VittSaathi is a **full-stack fintech application with React and Spring Boot 3 REST APIs**, uses **end-to-end JWT authentication, Redis caching, Docker, PostgreSQL**, and has a **Python/LangGraph/Gemini multi-agent financial advisory system with RAG**.

---

2. Project understanding questions

21. Why did you build VittSaathi?
22. What was the motivation behind the project?
23. Who are the intended users?
24. What are the major features?
25. What was your contribution?
26. Which part was the most challenging?
27. Which part did you build yourself?
28. What technologies did you use and why?
29. What alternatives did you consider?
30. What would you improve if you rebuilt the project?

A classic follow-up is:

"Why isn't VittSaathi just another chatbot?"

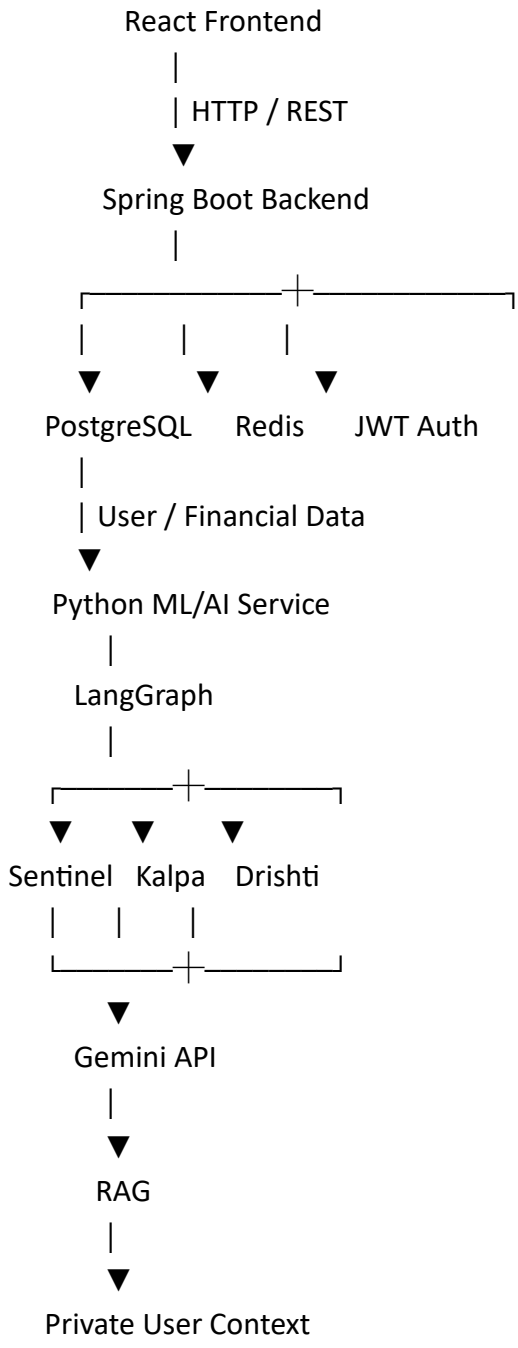
You should be able to explain the difference between a generic LLM chatbot and a **stateful financial advisory application that combines application data, retrieval, orchestration and specialized agents**.

---

3. Architecture questions

You absolutely need to know your architecture clearly.

A likely high-level structure is:



Then expect:

31. Why did you separate the Python AI service from Spring Boot?
32. Why not implement the agents directly in Java?
33. How does Spring Boot communicate with the Python service?
34. Why use microservice architecture here?
35. What are the advantages of separating services?
36. What are the disadvantages?
37. How do you handle failures between services?
38. How would you scale the Python service independently?
39. What happens if Gemini API is unavailable?
40. What happens if PostgreSQL is unavailable?
41. What happens if Redis is unavailable?

---

4. Spring Boot questions

Your resume explicitly says **Spring Boot 3 REST API**, so expect detailed backend questions.

42. What is Spring Boot?
43. Why Spring Boot instead of Spring Framework?
44. What is dependency injection?
45. What is IoC?
46. What is a Spring Bean?
47. What does @SpringBootApplication do?
48. What is auto-configuration?
49. What is component scanning?
50. Difference between @Component, @Service, and @Repository?
51. What is @RestController?
52. What is @RequestMapping?
53. Difference between @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping?
54. What is @RequestBody?
55. What is @PathVariable?
56. What is @RequestParam?
57. How do you handle exceptions in Spring Boot?
58. What is @ControllerAdvice?
59. How do you validate incoming requests?
60. What is DTO?
61. Why use DTOs instead of exposing entities directly?

---

5. REST API questions

Since your resume specifically mentions REST APIs, this is very likely.

62. What is REST?
63. What makes an API RESTful?
64. Difference between REST API and SOAP?
65. What are HTTP methods?
66. What is idempotency?
67. Which HTTP methods are idempotent?
68. What are HTTP status codes?
69. Difference between 200, 201, 400, 401, 403, 404 and 500?
70. What is statelessness in REST?
71. How did you structure your API endpoints?
72. How did you handle authentication in REST APIs?
73. How did you prevent unauthorized access?
74. How did you validate request payloads?

---

6. JWT authentication — VERY IMPORTANT

Your resume specifically claims **end-to-end JWT authentication**.

Prepare extremely well:

75. What is JWT?
76. Why did you use JWT?
77. How does JWT authentication work?
78. What are the three parts of a JWT?

79. What is the header?
80. What is the payload?
81. What is the signature?
82. Is JWT encrypted?
83. What is the difference between encoding and encryption?
84. How is a JWT generated?
85. How does the server validate a JWT?
86. Where do you store the JWT on the frontend?
87. What happens when the token expires?
88. What is refresh token?
89. Access token vs refresh token?
90. What is stateless authentication?
91. Why is JWT considered stateless?
92. Can a JWT be revoked?
93. What is the role of a secret key?
94. What happens if the secret key is compromised?
95. How do you protect JWTs against attacks?

Very common trap

"Is JWT more secure than sessions?"

Don't answer simply **yes**. Security depends on implementation, token storage, expiration, transport security, revocation strategy, CSRF/XSS exposure, etc.

---

7. PostgreSQL questions

Your resume explicitly mentions **version-controlled PostgreSQL**.

96. Why PostgreSQL?
97. Why not MySQL?
98. What is a relational database?
99. What is a primary key?
100. What is a foreign key?
101. What is normalization?
102. Explain 1NF, 2NF and 3NF.
103. What is an index?
104. How does an index improve performance?
105. What are the disadvantages of indexes?
106. What is a transaction?
107. Explain ACID properties.
108. What is a JOIN?
109. Difference between INNER JOIN and LEFT JOIN?
110. What is a database constraint?
111. How did you design the database schema?
112. How are users and their financial data related?
113. How do you prevent SQL injection?
114. How do you manage database migrations/versioning?

---

8. Redis questions

Your resume says **Redis-backed caching**, so the interviewer can easily drill into this.

115. What is Redis?
116. Why use Redis?
117. Why not directly query PostgreSQL every time?
118. What is caching?
119. Explain cache-aside strategy.
120. What data did you cache?
121. When is caching useful?
122. What happens when cached data becomes stale?
123. What is cache invalidation?
124. What is TTL?
125. What happens when a key expires?
126. What happens if Redis goes down?
127. Can Redis be used as a primary database?
128. Difference between Redis and PostgreSQL?
129. What Redis data structures do you know?
130. How does Redis achieve high performance?

---

9. Docker questions

You mention Docker explicitly.

131. What is Docker?
132. Why did you use Docker?
133. What is a container?
134. Container vs virtual machine?
135. What is a Docker image?
136. What is a Dockerfile?
137. What is Docker Compose?
138. How did you containerize your application?
139. How did you connect containers?
140. How did you manage environment variables?
141. Why shouldn't secrets be hardcoded in Dockerfiles?
142. How would you deploy this system to production?

---

10. LangGraph — HIGH PRIORITY

This will probably be one of the most interesting parts of your interview.

Your resume says you built a **stateful multi-agent financial advisory system using LangGraph and Gemini API**, with three specialized agents and a shared **StateGraph**.

143. What is LangGraph?
144. Why LangGraph instead of LangChain alone?
145. What is a graph in LangGraph?
146. What is a node?
147. What is an edge?
148. What is a state?
149. What is StateGraph?
150. Why is state important in your application?

151. How does information move between agents?
152. What does "cyclic workflow" mean?
153. Why did you need a cyclic workflow?
154. How do you prevent infinite loops?
155. How do you decide which agent runs next?
156. What happens if one agent fails?
157. Why use multiple agents instead of one agent?
158. What are the responsibilities of Sentinel, Kalpa and Drishti?
159. What does each agent receive as input?
160. What does each agent produce as output?
161. How do you maintain shared context?
162. How do you control the agents' behavior?

---

11. Multi-agent system questions

Expect the interviewer to challenge the design choice:

"Why three agents? Why not just send the entire prompt to Gemini?"

Then:

163. What is a multi-agent system?
164. What is the advantage of specialization?
165. What are the disadvantages of multiple agents?
166. Does using more agents increase latency?
167. Does it increase cost?
168. How do you prevent agents from producing conflicting answers?
169. Who makes the final decision?
170. How is state shared?
171. Can agents execute in parallel?
172. Why are your agents cyclic rather than linear?
173. How would you monitor agent execution?
174. How would you debug an incorrect final response?

---

12. Gemini API questions

175. What is Gemini?
176. Why did you choose Gemini?
177. How does your application communicate with Gemini?
178. What is a prompt?
179. What is prompt engineering?
180. What is temperature?
181. What is hallucination?
182. How do you reduce hallucination?
183. How do you ensure the model doesn't invent financial data?
184. How do you control the structure of the output?
185. What happens if Gemini returns an invalid response?
186. How do you handle API failures?
187. How do you handle API rate limits?
188. How do you protect your Gemini API key?

---

13. RAG — EXTREMELY IMPORTANT

Your resume says you developed a **vector-based RAG system** to give Gemini **long-term memory and real-time access to private user data**.

Expect:

189. What is RAG?
190. Why do we need RAG?
191. How does RAG work?
192. What is an embedding?
193. What is a vector embedding?
194. What is a vector database?
195. What is semantic search?
196. What is cosine similarity?
197. What is chunking?
198. Why do we chunk documents/data?
199. What is the retrieval phase?
200. What is the generation phase?
201. How does retrieved data reach Gemini?
202. How does RAG reduce hallucination?
203. RAG vs fine-tuning?
204. Why didn't you fine-tune Gemini?
205. How does RAG provide long-term memory?
206. How do you retrieve the correct user's data?
207. How do you prevent one user's financial data from being retrieved for another user?
208. How do you update embeddings when user data changes?
209. What happens if retrieval returns irrelevant information?
210. How do you evaluate retrieval quality?

---

14. Security of private financial data

This could become a **major interview topic** because your resume claims access to private user data.

211. How do you isolate users' data?
212. How do you ensure user A cannot access user B's data?
213. Where does authorization happen?
214. Does JWT authentication alone provide authorization?
215. How do you enforce ownership of financial records?
216. How do you secure data between Spring Boot and Python?
217. How do you secure PostgreSQL?
218. How do you protect Redis?
219. How do you protect API keys?
220. How would you encrypt sensitive data?
221. How would you audit access to financial data?
222. What would happen if your vector database is compromised?

---

15. React questions

Since React is part of your full-stack architecture:

223. What is React?
224. What is a component?
225. What is JSX?
226. What are props?
227. What is state?
228. Props vs state?
229. What is useState?
230. What is useEffect?
231. What is the Virtual DOM?
232. How does React communicate with Spring Boot?
233. How do you store authentication state?
234. How do you handle API errors?
235. How do you protect frontend routes?
236. What happens when the JWT expires?

---

16. Full-stack integration questions

237. Explain the complete login flow.

For example:

React
↓
POST /login
↓
Spring Boot
↓
Authenticate user
↓
Generate JWT
↓
Return token
↓
React stores token
↓
Token sent with future requests
↓
Spring Security validates token
↓
Request reaches controller

Then they may ask:

238. Explain a complete financial-data request.
239. Where does authentication happen?
240. Where does authorization happen?
241. Where does caching happen?
242. Where does PostgreSQL come into the flow?
243. When is the Python service called?
244. When is Gemini called?

245. When is RAG used?

---

17. Advanced questions

For a stronger interviewer:

246. How would you handle concurrent requests?
247. How would you make your application horizontally scalable?
248. What is load balancing?
249. What happens when two requests update the same financial record?
250. How do transactions prevent inconsistent data?
251. How would you design idempotent APIs?
252. How would you monitor the system?
253. How would you implement logging?
254. How would you trace a request across Spring Boot → Python → Gemini?
255. How would you handle retries?
256. How would you implement circuit breakers?
257. How would you handle Gemini rate limits?
258. How would you reduce LLM latency?
259. How would you reduce LLM cost?
260. How would you handle malicious prompts/user input?
261. How would you prevent prompt injection against your RAG system?

---

18. Questions comparing technologies

These are extremely common in placements:

Spring Boot vs Node.js

- Why Spring Boot?

PostgreSQL vs MongoDB

- Why PostgreSQL for financial data?

Redis vs PostgreSQL

- Why do you need Redis when PostgreSQL already stores the data?

JWT vs Session Authentication

- Why JWT?

REST vs WebSocket

- Why REST for this application?

RAG vs Fine-tuning

- Why RAG?

One LLM agent vs Multi-agent

- Why three agents?

LangGraph vs LangChain

- Why LangGraph?

Python vs Java

- Why is your AI service in Python while your main backend is Java?

That last question is particularly likely because your resume explicitly shows **Spring Boot + a Python microservice for the AI layer**.

---

19. Top 25 questions to master

If you're short on preparation time, these are the **25 I would absolutely prepare**:

Priority	Question
1	Explain VittSaathi
2	Explain complete architecture
3	Explain end-to-end request flow
4	Why Spring Boot?
5	What is REST API?
6	Explain JWT authentication
7	JWT vs session authentication
8	How do you implement authorization?
9	Why PostgreSQL?
10	Why Redis?
11	What is caching?
12	How does cache invalidation work?
13	Why Docker?
14	Why Python microservice?
15	What is LangGraph?
16	What is StateGraph?
17	Why three agents?
18	Explain Sentinel, Kalpa and Drishti
19	What is RAG?
20	How does your RAG pipeline work?
21	RAG vs fine-tuning
22	How do you protect private user data?
23	How does Gemini fit into the system?
24	What happens if Gemini/API/database fails?
25	What would you improve in VittSaathi?